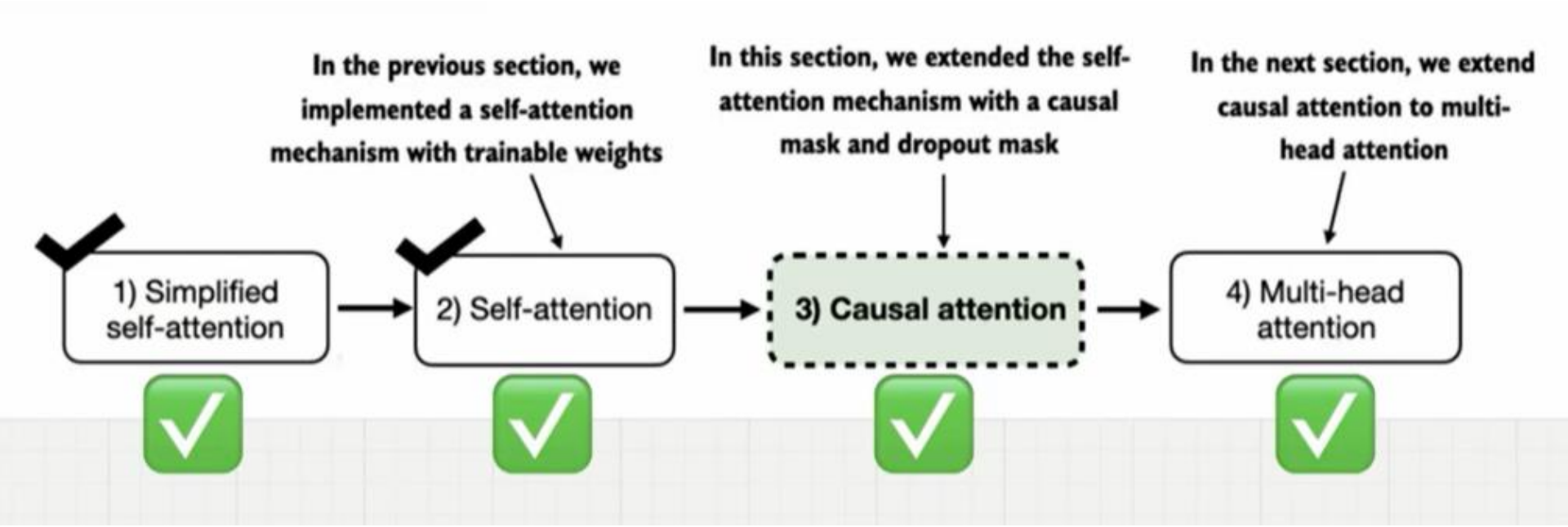
Yazılım Mühendisliği Bölümü

YZM358 Doğal Dil İşleme
2025-2026 Bahar Yarıyılı

Hafta 9 LLM mimarisine Bakış

Doç. Dr. Musa AYDIN

Geçmiş haftalarda:



Bu hafta:

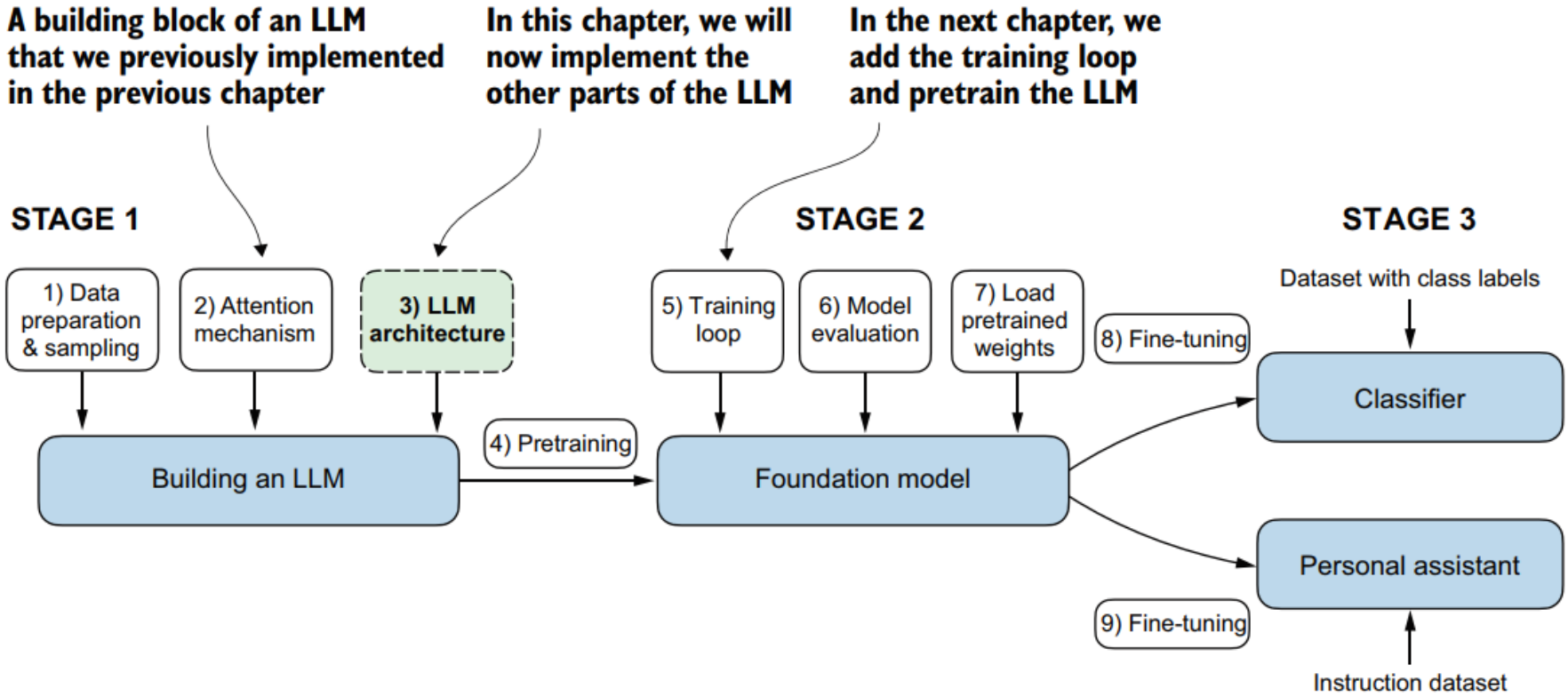
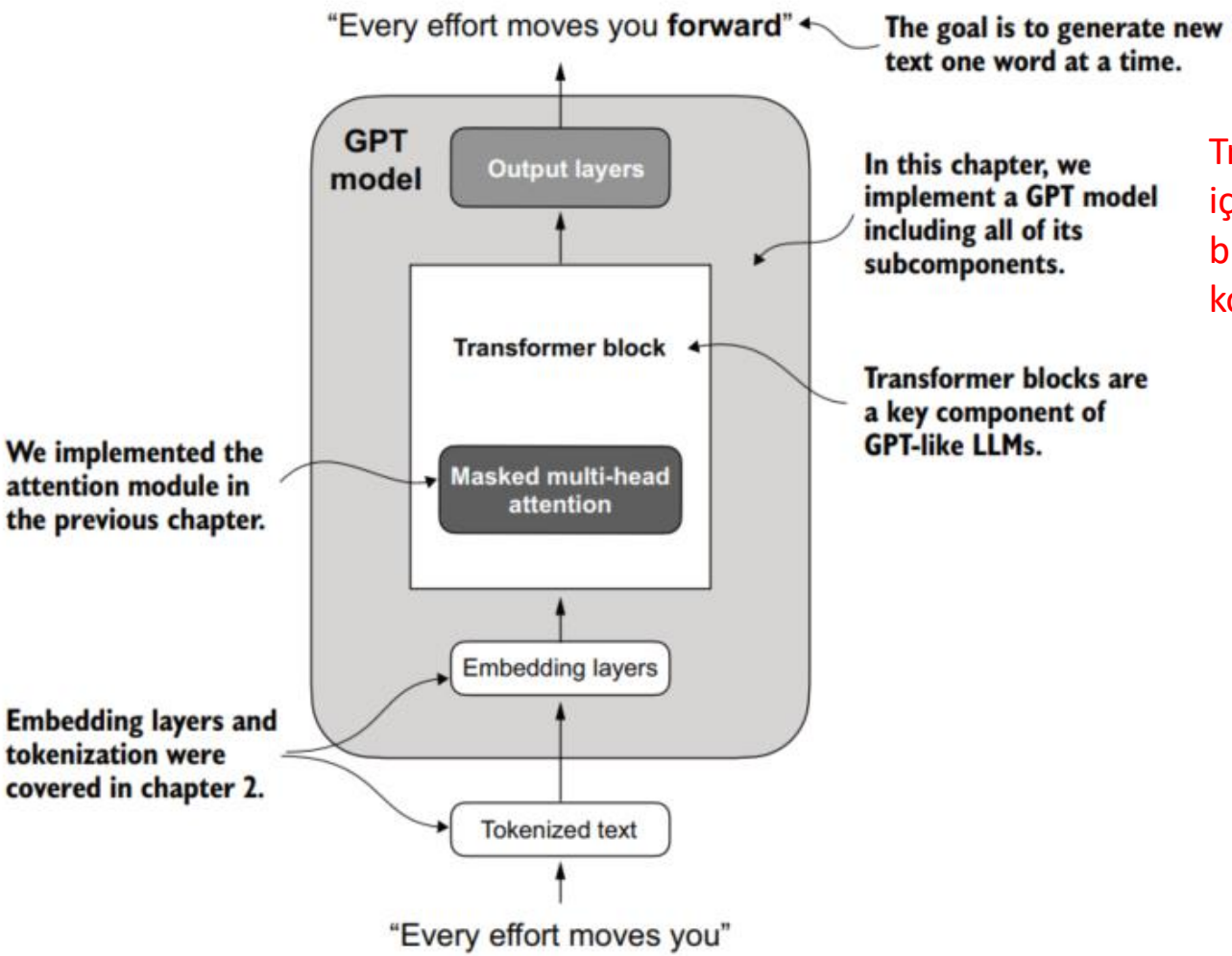


Figure 4.1 The three main stages of coding an LLM. This chapter focuses on step 3 of stage 1: implementing the LLM architecture.

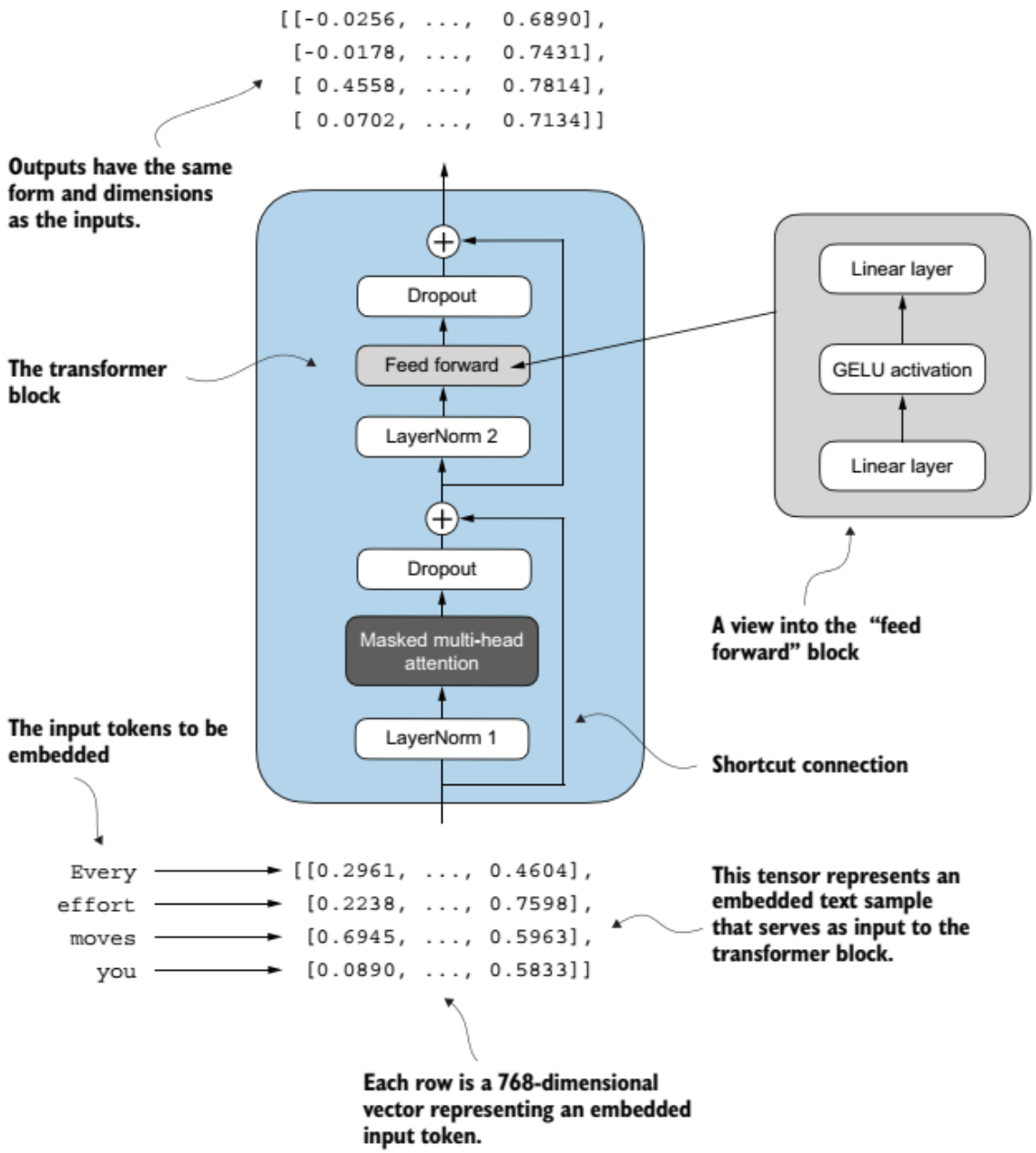
LLM architecture



Transformer bloğu içerisindeki kalan blokları kodlayacağız

Figure 4.2 A GPT model. In addition to the embedding layers, it consists of one or more transformer blocks containing the masked multi-head attention module we previously implemented.

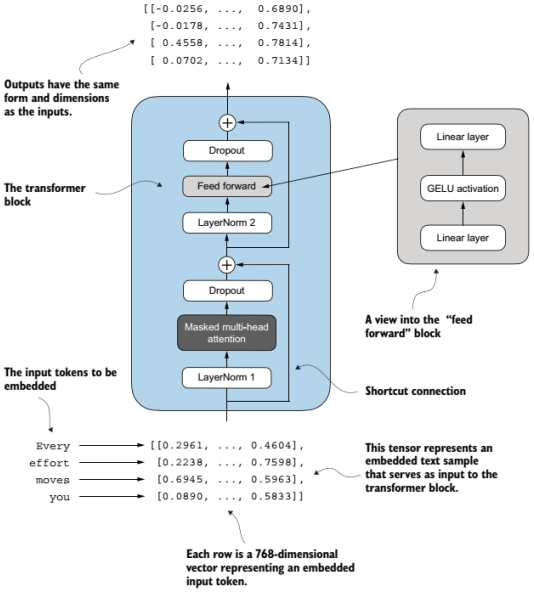
LLM architecture



Tek bir transformer head içerisindeki bloklar-modüller

LLM architecture – şu anda kuş bakışı bakıyoruz transformer' a

Tek bir transformer head içerisindeki bloklar-modüller



LLM architecture

Bu zamana kadar ne öğrendik:

1. Input tokenization
2. Embedding (Token-Vector Embedding + Positional Encoding)
3. Masked Multi-head Attention

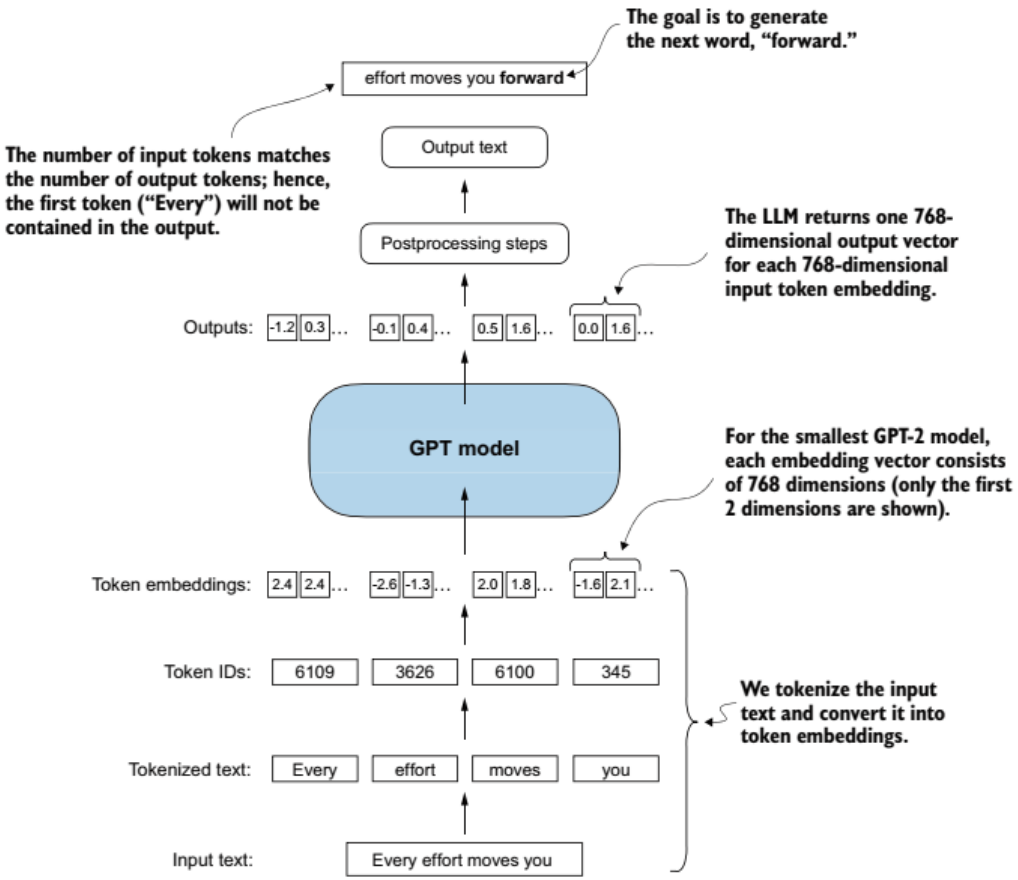


Figure 4.4 A big-picture overview showing how the input data is tokenized, embedded, and fed to the GPT model. Note that in our `DummyGPTClass` coded earlier, the token embedding is handled inside the GPT model. In LLMs, the embedded input token dimension typically matches the output dimension. The output embeddings here represent the context vectors (see chapter 3).

LLM architecture

Bu zamana kadar ne öğrendik:

1. Input tokenization
2. Embedding (Token-Vector Embedding + Positional Encoding)
3. Masked Multi-head Attention

Çalışmalarımızda GPT2 modelini kullanacağız, çünkü bu modelin eğitilmiş ağırlıkları paylaşılmaktadır.

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

Table 2. Architecture hyperparameters for the 4 model

LLM architecture

Bu zamana kadar ne öğrendik:

1. Input tokenization
2. Embedding (Token-Vector Embedding + Positional Encoding)
3. Masked Multi-head Attention

Çalışmalarımızda GPT2 modelini kullanacağız, çünkü bu modelin eğitilmiş ağırlıkları paylaşılmaktadır. Aşağıdaki dictionary objesinde GPT2 için configürasyon parametreleri görülmektedir.

We specify the configuration of the small GPT-2 model via the following Python dictionary, which we will use in the code examples later:

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,     # Vocabulary size
    "context_length": 1024,  # Context length
    "emb_dim": 768,          # Embedding dimension
    "n_heads": 12,           # Number of attention heads
    "n_layers": 12,          # Number of layers
    "drop_rate": 0.1,        # Dropout rate
    "qkv_bias": False        # Query-Key-Value bias
}
```

LLM architecture

Bu zamana kadar ne öğrendik:

1. Input tokenization
2. Embedding (Token-Vector Embedding + Positional Encoding)
3. Masked Multi-head Attention

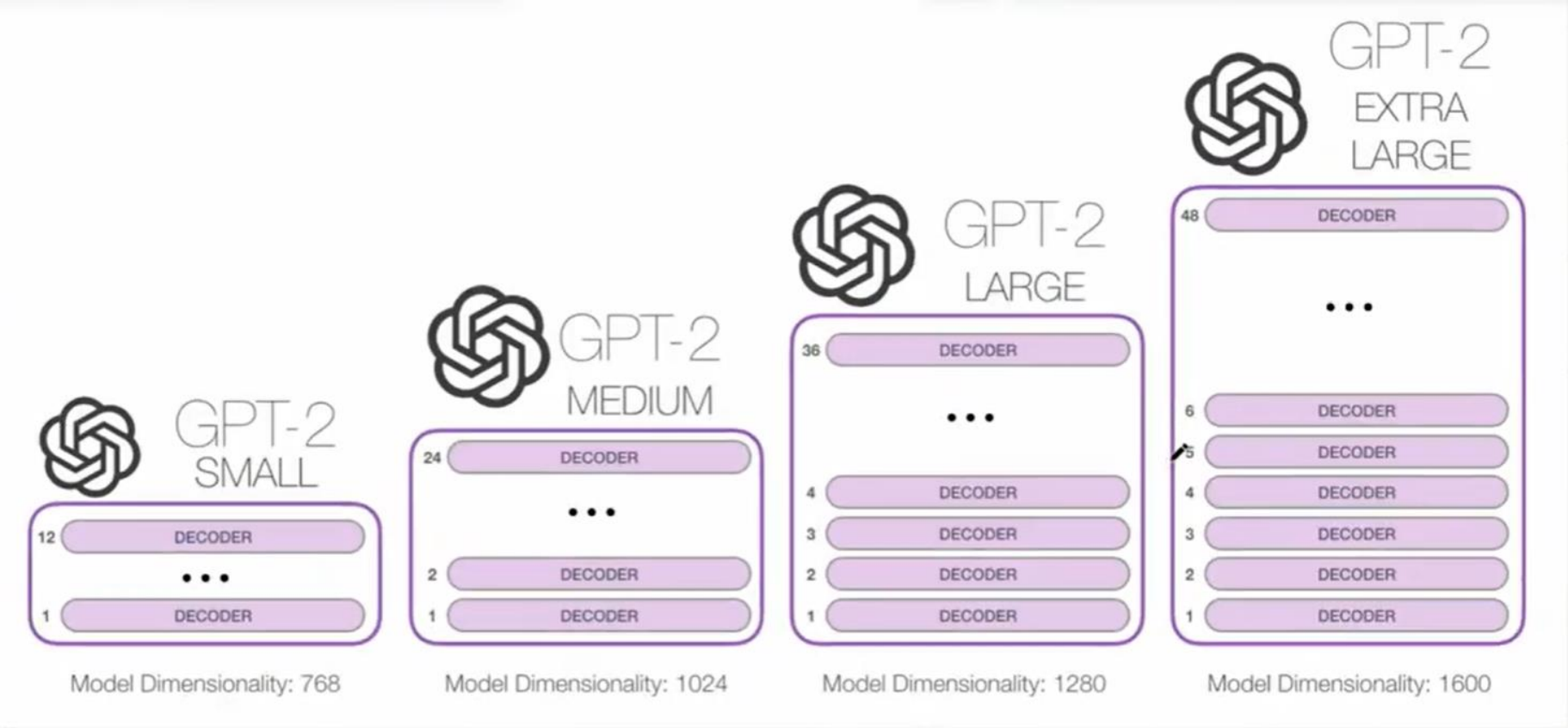
Çalışmalarımızda GPT2 modelini kullanacağız, çünkü bu modelin eğitilmiş ağırlıkları paylaşılmaktadır. Aşağıdaki dictionary objesinde GPT2 için configürasyon parametreleri görülmektedir.

In the `GPT_CONFIG_124M` dictionary, we use concise variable names for clarity and to prevent long lines of code:

- `vocab_size` refers to a vocabulary of 50,257 words, as used by the BPE tokenizer (see chapter 2).
- `context_length` denotes the maximum number of input tokens the model can handle via the positional embeddings (see chapter 2).
- `emb_dim` represents the embedding size, transforming each token into a 768-dimensional vector.
- `n_heads` indicates the count of attention heads in the multi-head attention mechanism (see chapter 3).
- `n_layers` specifies the number of transformer blocks in the model, which we will cover in the upcoming discussion.
- `drop_rate` indicates the intensity of the dropout mechanism (0.1 implies a 10% random drop out of hidden units) to prevent overfitting (see chapter 3).
- `qkv_bias` determines whether to include a bias vector in the `Linear` layers of the multi-head attention for query, key, and value computations. We will initially disable this, following the norms of modern LLMs, but we will revisit it in chapter 6 when we load pretrained GPT-2 weights from OpenAI into our model (see chapter 6).

LLM architecture

Çalışmalarımızda GPT2 modelini kullanacağız, çünkü bu modelin eğitilmiş ağırlıkları paylaşılmaktadır. Aşağıdaki GPT2 için farklı boyutlardaki versiyonlarının mimari özellikleri görülmektedir.



LLM architecture

Basit GPT model kodu-hazırlık

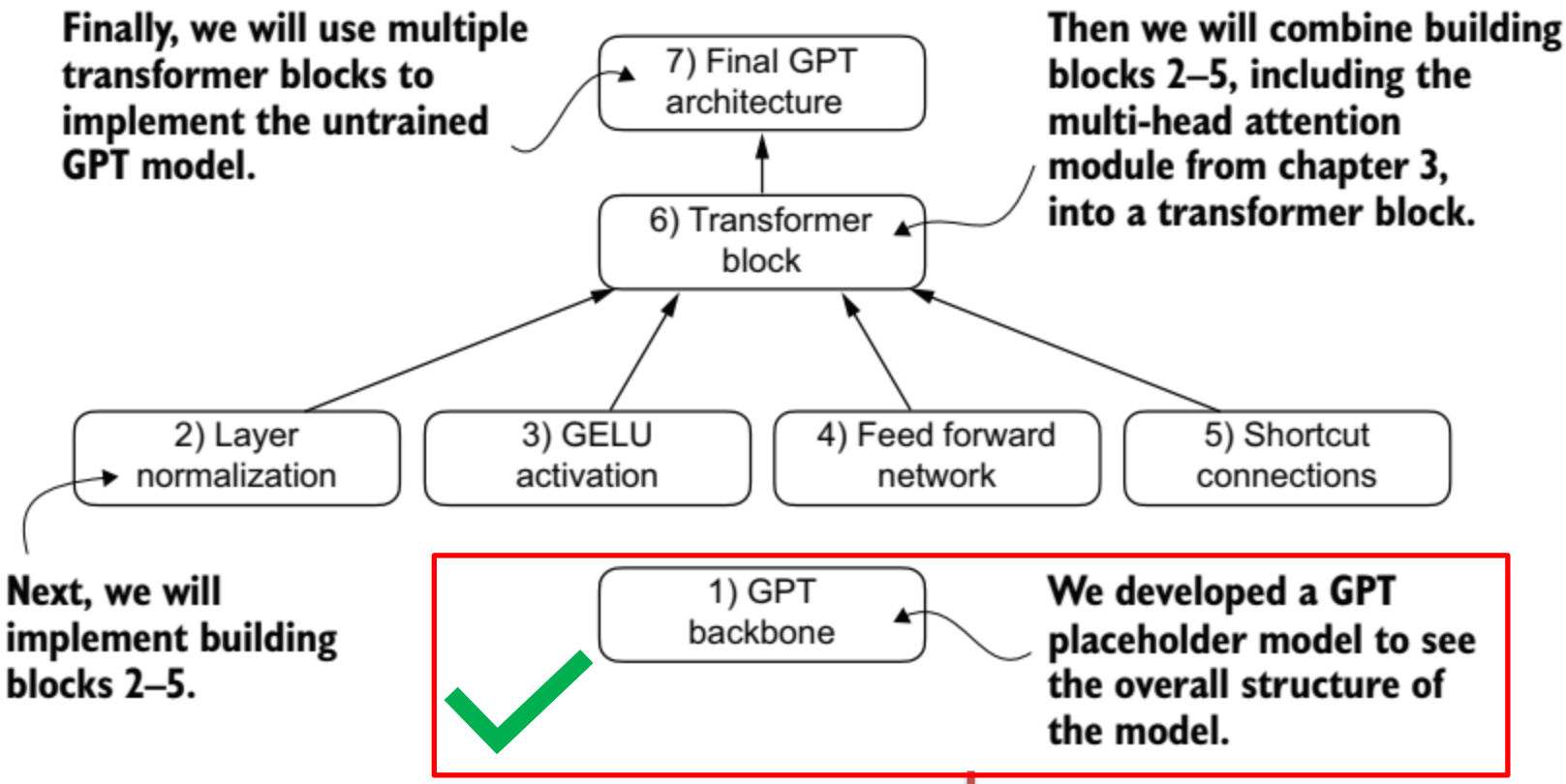


Figure 4.3 The order in which we code the GPT architecture. We start with the GPT backbone, a placeholder architecture, before getting to the individual core pieces and eventually assembling them in a transformer block for the final GPT architecture.

İlk olarak GPT modeli genel yapısını kodlayalım.

LLM architecture

Basit GPT model kodu-hazırlık

```
import tiktoken

tokenizer = tiktoken.get_encoding("gpt2")
batch = []
txt1 = "Every effort moves you"
txt2 = "Every day holds a"

batch.append(torch.tensor(tokenizer.encode(txt1)))
batch.append(torch.tensor(tokenizer.encode(txt2)))
batch = torch.stack(batch, dim=0)
print(batch)
```

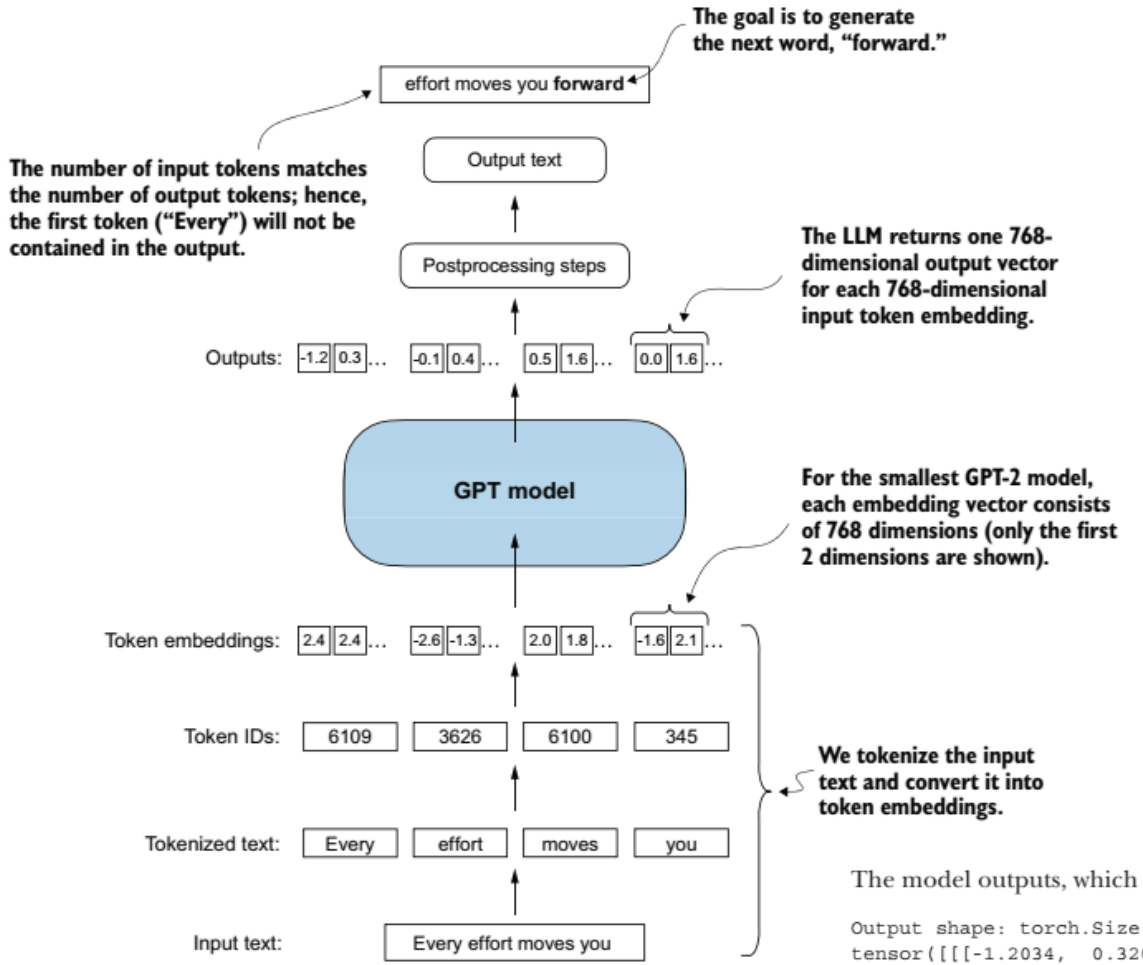
Output:
tensor([[6109, 3626, 6100, 345],
[6109, 1110, 6622, 257]])

Model sınıfında tanımlanan token embedding table ile, positional embedding table size nedir ? Neden ?

```
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
```

LLM architecture

Basit GPT model kodu-hazırlık



The model outputs, which are commonly referred to as logits, are as follows:

```
Output shape: torch.Size([2, 4, 50257])
tensor([[[-1.2034,  0.3201, -0.7130,  ..., -1.5548, -0.2390, -0.4667],
         [-0.1192,  0.4539, -0.4432,  ...,  0.2392,  1.3469,  1.2430],
         [ 0.5307,  1.6720, -0.4695,  ...,  1.1966,  0.0111,  0.5835],
         [ 0.0139,  1.6755, -0.3388,  ...,  1.1586, -0.0435, -1.0400]],

        [[-1.0908,  0.1798, -0.9484,  ..., -1.6047,  0.2439, -0.4530],
         [-0.7860,  0.5581, -0.0610,  ...,  0.4835, -0.0077,  1.6621],
         [ 0.3567,  1.2698, -0.6398,  ..., -0.0162, -0.1296,  0.3717],
         [-0.2407, -0.7349, -0.5102,  ...,  2.0057, -0.3694,  0.1814]]],
       grad_fn=<UnsafeViewBackward0>)
```

LLM architecture-Layer Normalization

Basit GPT model kodu-hazırlık

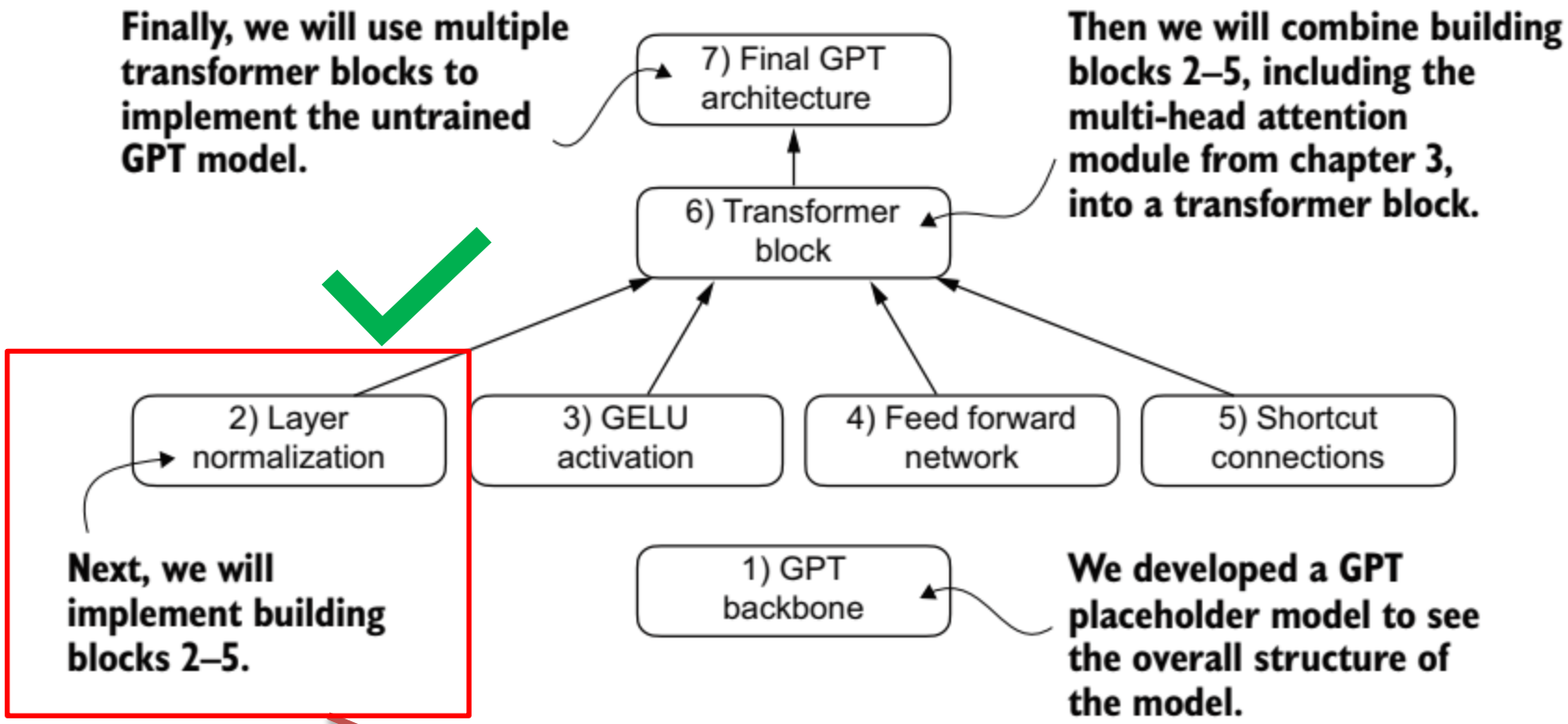


Figure 4.3 The order in which we code the GPT architecture. We start with the GPT backbone, a placeholder architecture, before getting to the individual core pieces and eventually assembling them in a transformer block for the final GPT architecture.

Katman normalizasyon işleminin nasıl yapıldığı anlatılacaktır.

LLM architecture-Layer Normalization

Layer Normalization (Katman Normalizasyonu):

Nedir: Bir sinir ağında (özellikle Transformer'larda) bir katmandan çıkan verilerin (embedding vektörlerinin) bir sonraki katmana girmeden önce ortalamasının 0'a, varyansının (dağılımının) 1'e çekilmesi işlemidir.

Ne İşe Yarar? (Neden Kullanılır?): Bir ağ derinleştikçe, arka arkaya yapılan matris çarpımları ve toplamaları yüzünden sayılar kontrolden çıkabilir. Bazı özellikler (feature) binlerce değerine ulaşırken, bazıları sıfıra çok yaklaşabilir. Buna **"Covariate Shift" (İçsel Ortam Kayması) denir.** Eğer değerler çok büyürse, modelin ağırlıklarını güncelleyen gradyanlar da çok büyür **(Exploding Gradient) veya çok küçülür (Vanishing Gradient)** ve model öğrenmeyi unutup rastgele değerler üretir-öğrenme çok minik oranlarda gerçekleşir. **Layer Norm**, bu sayıları sürekli ehlileştirerek-düzenleyerek aynı hizada ve ölçekte tutar.

LLM architecture-Layer Normalization

Layer Normalization (Katman Normalizasyonu):

Nedir: Bir sinir ağında (özellikle Transformer'larda) bir katmandan çıkan verilerin (embedding vektörlerinin) bir sonraki katmana girmeden önce ortalamasının 0'a, varyansının (dağılımının) 1'e çekilmesi işlemidir.

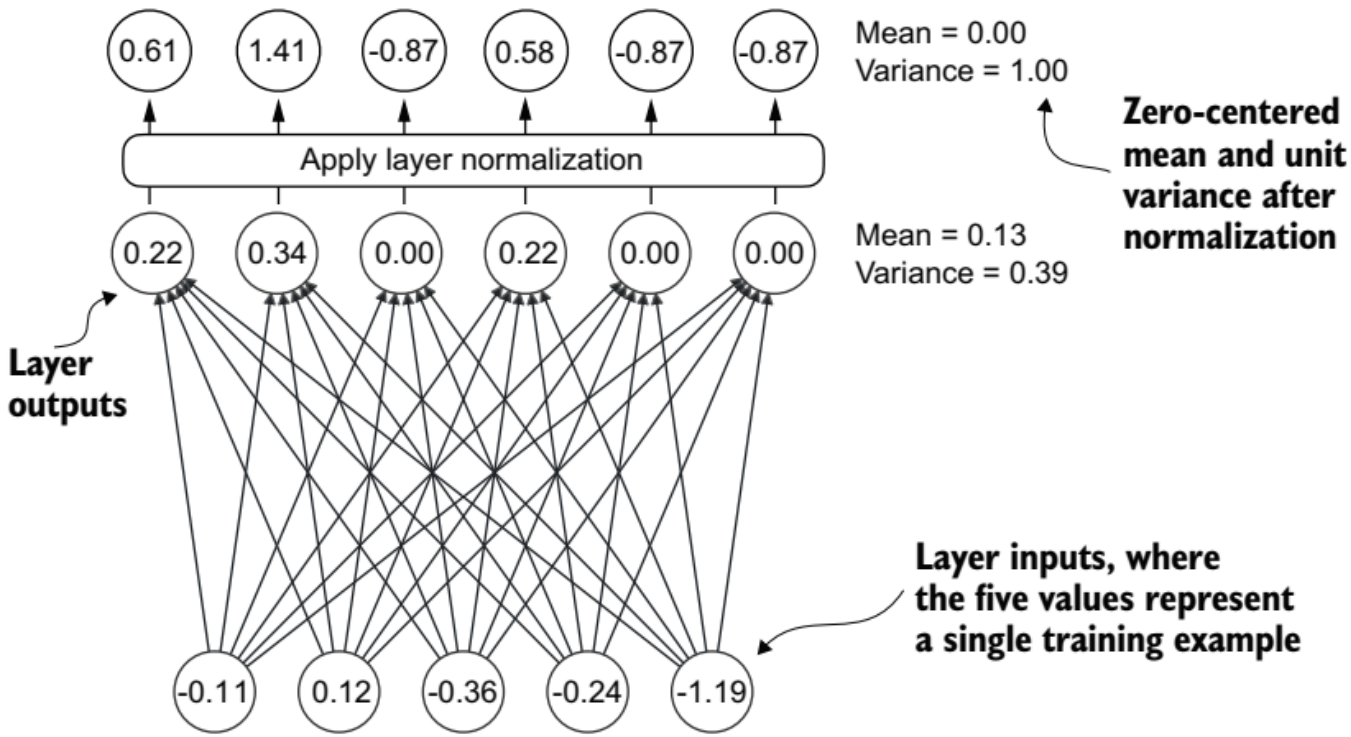


Figure 4.5 An illustration of layer normalization where the six outputs of the layer, also called activations, are normalized such that they have a 0 mean and a variance of 1.

LLM architecture-Layer Normalization

Layer Normalization (Katman Normalizasyonu):

```
torch.manual_seed(123)
batch_example = torch.randn(2, 5)
layer = nn.Sequential(nn.Linear(5, 6), nn.ReLU())
out = layer(batch_example)
print(out)
```

Creates two training examples with five dimensions (features) each

This prints the following tensor, where the first row lists the layer outputs for the f input and the second row lists the layer outputs for the second row:

```
tensor([[0.2260, 0.3470, 0.0000, 0.2216, 0.0000, 0.0000],
        [0.2133, 0.2394, 0.0000, 0.5198, 0.3297, 0.0000]],
       grad_fn=<ReluBackward0>)
```

```
mean = out.mean(dim=-1, keepdim=True)
var = out.var(dim=-1, keepdim=True)
print("Mean:\n", mean)
print("Variance:\n", var)
```

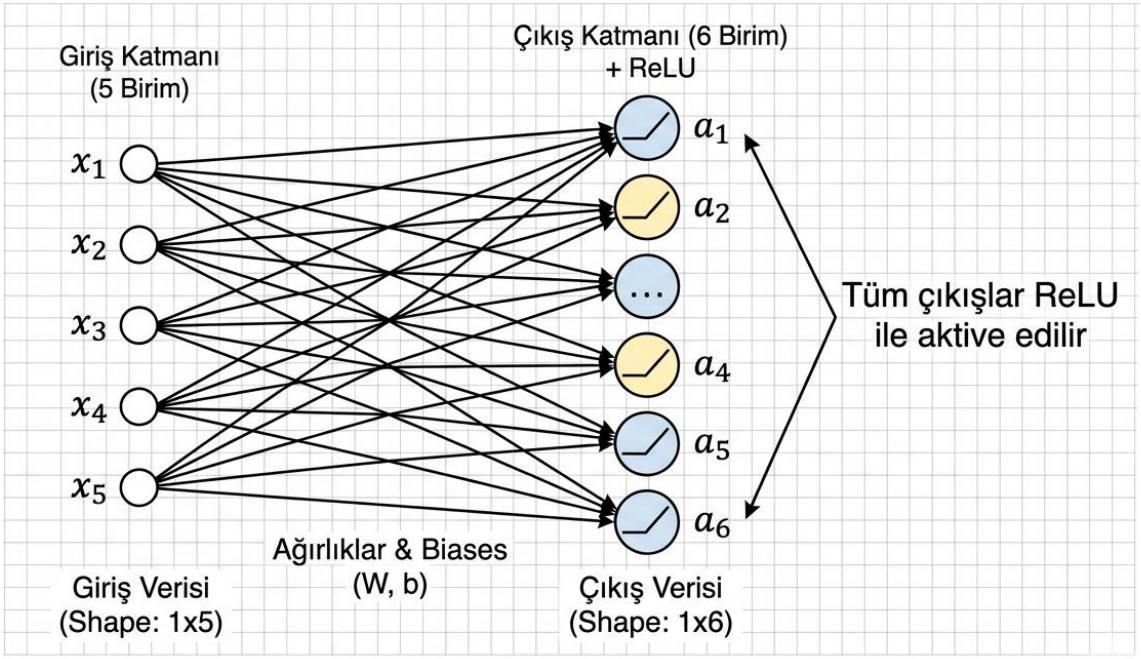
The output is

```
Mean:
  tensor([[0.1324],
          [0.2170]], grad_fn=<MeanBackward1>)
Variance:
  tensor([[0.0231],
          [0.0398]], grad_fn=<VarBackward0>)
```

İlk batch çıktı değerleri

İkinci batch çıktı değerleri

Figure 1: Basit 5x6 MLP Mimarisi ve ReLU Aktivasyonu



ReLu Activation Function

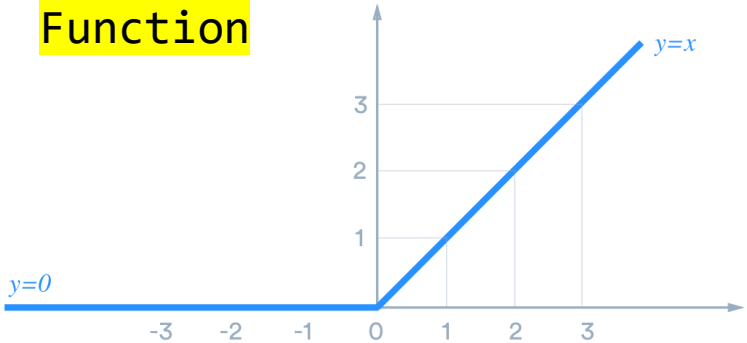


Figure 1: Basit 5x6 MLP Mimarisi ve ReLU Aktivasyonu

LLM architecture-GELU activation function

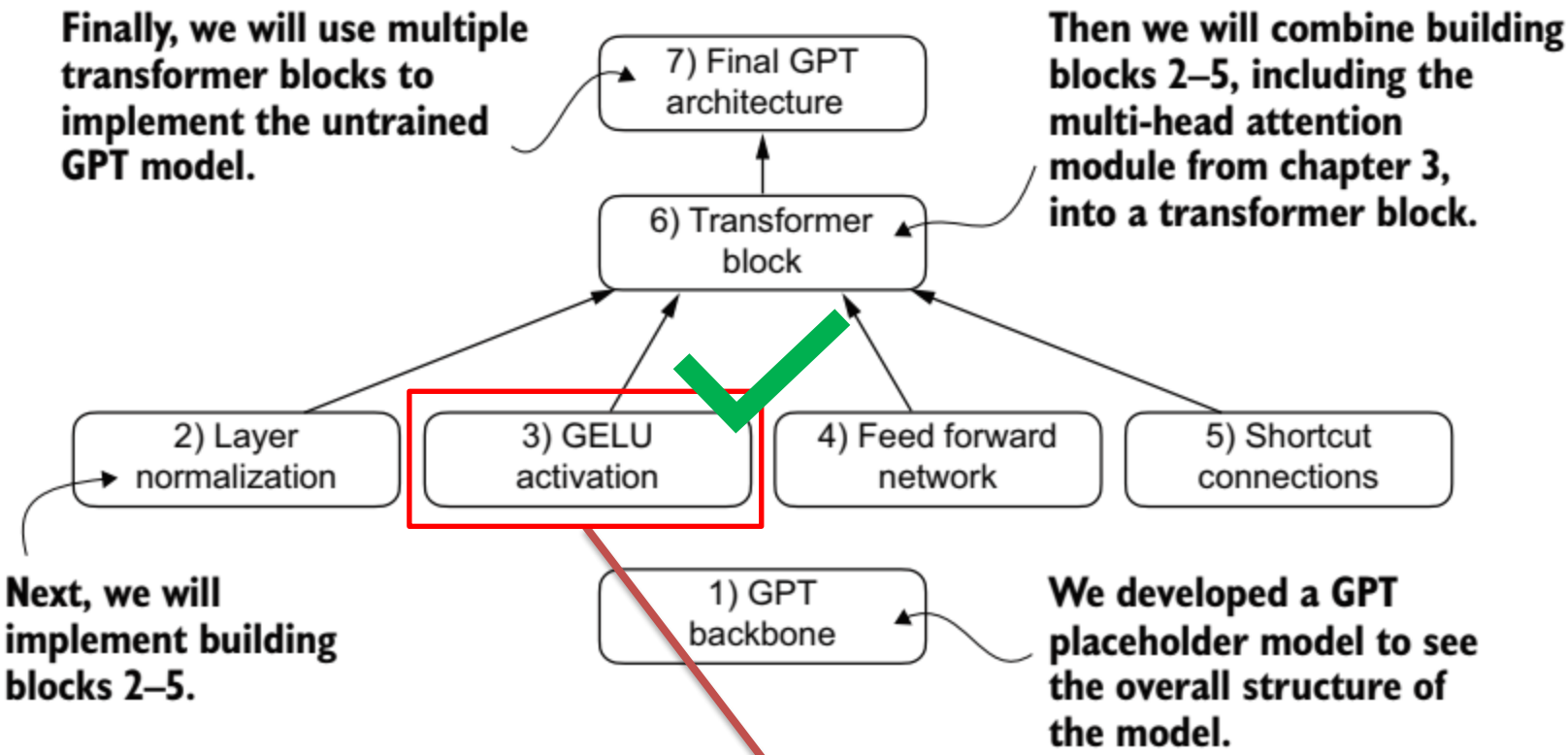


Figure 4.3 The order in which we code the GPT architecture. We start with the GPT backbone, a placeholder architecture, before getting to the individual core pieces and eventually assembling them in a transformer block for the final GPT architecture.

GeLu Activation Function

LLM architecture-GELU activation function

GeLU ((Gaussian Error Linear Unit)) aktivasyon fonksiyonu:

Aktivasyon Fonksiyonu Nedir? (Doğrusallığı Kırmak):

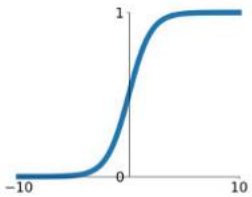
Sinir ağlarındaki ağırlıkların (weights) matris çarpımları ve toplamaları sadece doğrusal (linear) işlemlerdir.

Eğer araya bir aktivasyon fonksiyonu koymazsak, 100 katmanlı bir ağ bile matematiksel olarak tek katmanlı basit bir doğrusal denkleme (y = mx + b) indirgenir.

Aktivasyon fonksiyonları, ağa kıvrım (non-linearity) katar. Nöronlara 'Bu bilgiyi geçir', 'Bunu engelle' veya 'Bunu biraz törpüle' diyerek onlara bir karar verme yeteneği kazandırır.

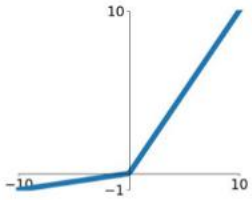
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



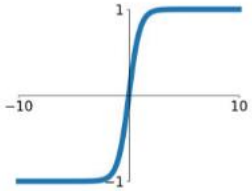
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

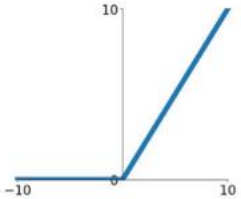


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

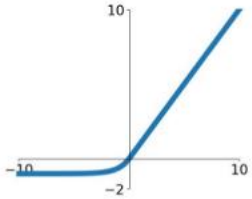
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



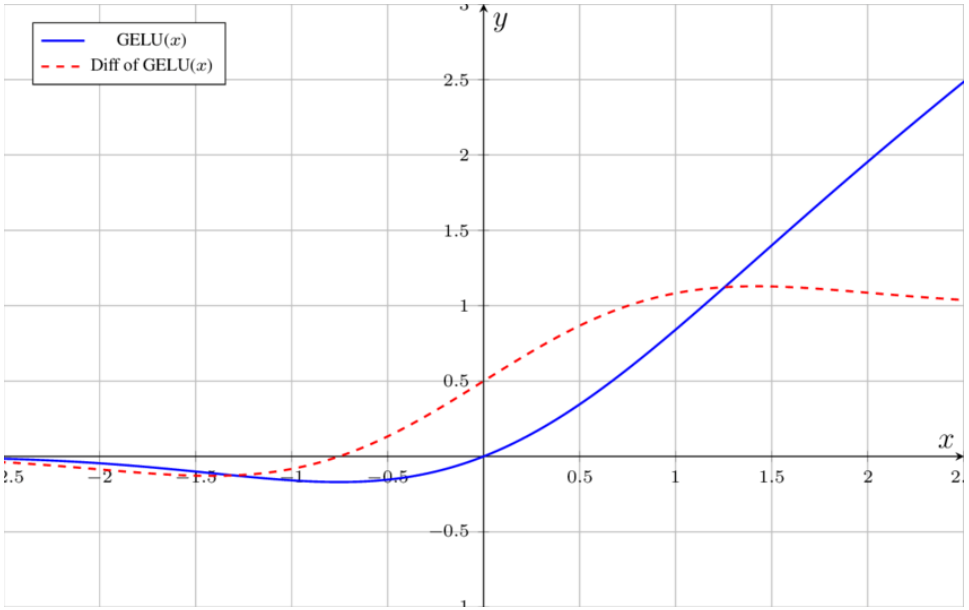
LLM architecture-GELU activation function

GeLU ((Gaussian Error Linear Unit)) aktivasyon fonksiyonu:

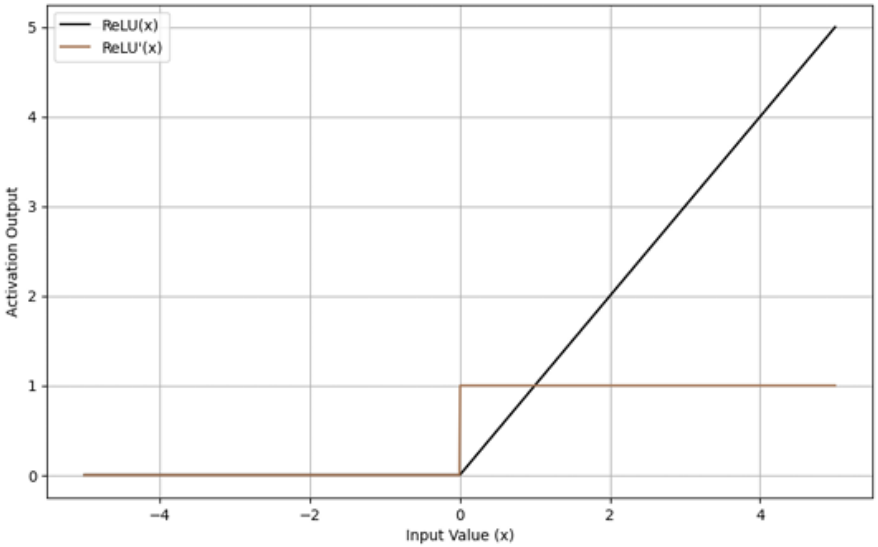
GELU (Gaussian Error Linear Unit) ise bu kapının önüne bir olasılık dağılımı (Gaussian/Normal Dağılım) koyar. Keskin bir köşesi yoktur, yumuşak bir kavis çizer.

- Sıfırın hemen altındaki negatif değerlere (örn: -0.5) "tamamen öl" demez, onlara küçük bir negatif değerle geçiş izni verir.
- Derin negatiflerde sıfıra yaklaşır.
- Pozitiflerde ise ReLU gibi davranarak sayıyı aynen geçirir.

GeLU-Türevi



Relu-Türevi



LLM architecture-GELU activation function

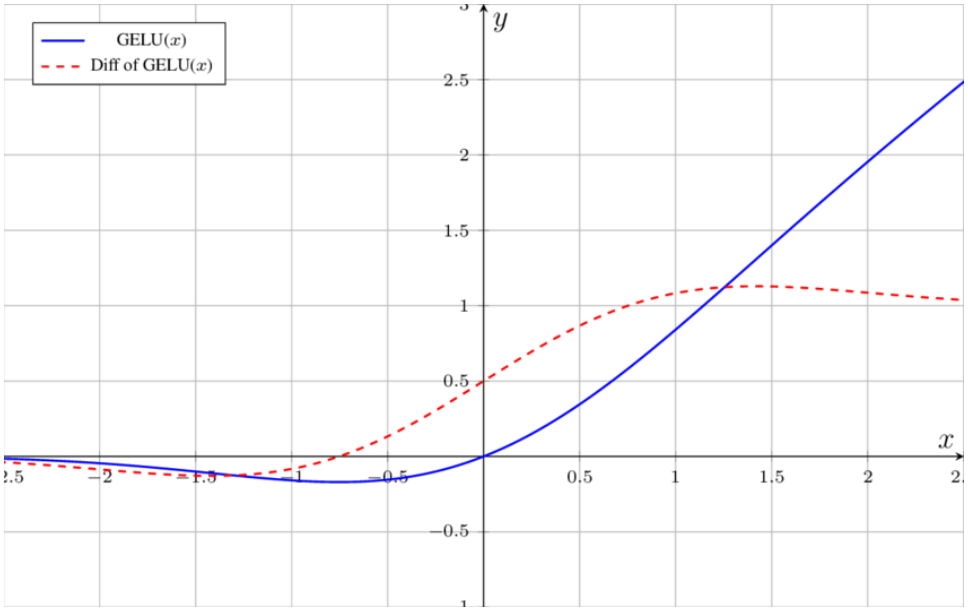
GeLU ((Gaussian Error Linear Unit)) aktivasyon fonksiyonu:

Pürüzsüz Gradyanlar (Ölü Nöron Probleminin Çözümü): ReLU'nun sıfır noktasında sert bir "V" köşesi vardır (türevi tanımsızdır) ve **0'dan küçük her şeyin türevi anında 0 olur (Dying ReLU - Ölü Nöron problemi).**

Modelin o kısmı öğrenmeyi durdurur. GELU'nun o yumuşak kavisi (her yerde türevlenebilir olması), backpropagation (geriye yayılım) sırasında hata sinyallerinin (gradyanların) çok daha pürüzsüz, kesintisiz ve sağlıklı akmasını sağlar. Transformer'lar gibi devasa ve derin modellerde bu pürüzsüzlük, modelin çökmesini engeller.

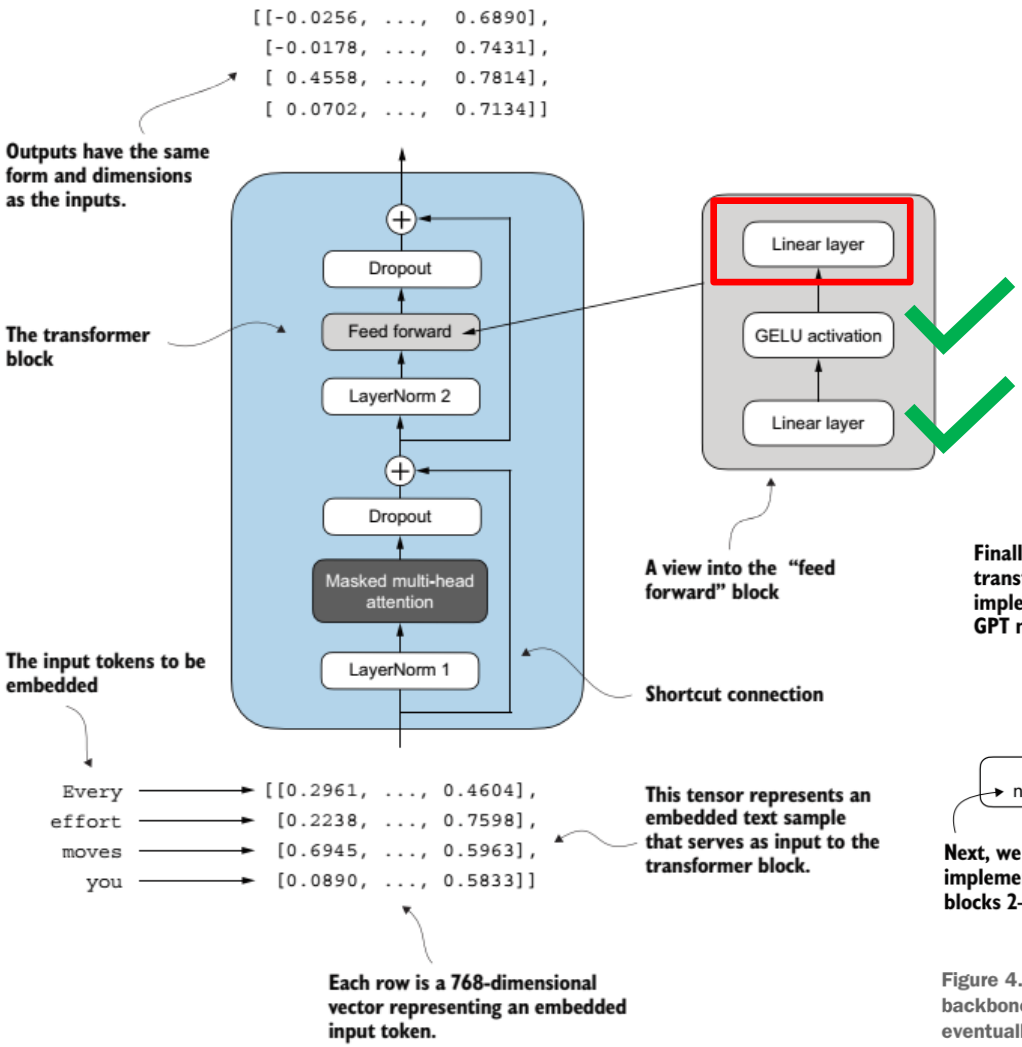
$$GELU(x) = 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

GeLU-Türevi



LLM architecture-Feed Forward Neural Network

FFN (Feed Forward Neural Network)



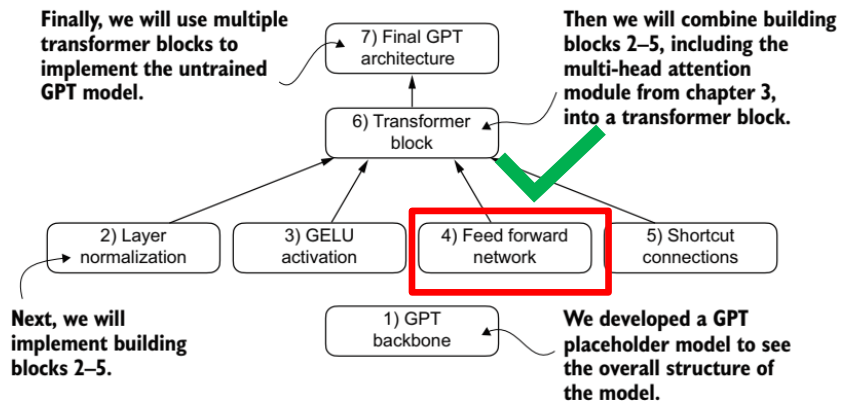


Figure 4.3 The order in which we code the GPT architecture. We start with the GPT backbone, a placeholder architecture, before getting to the individual core pieces and eventually assembling them in a transformer block for the final GPT architecture.

LLM architecture-Feed Forward Neural Network

FFN (Feed Forward Neural Network):

- **İleri beslemeli sinir ağları:** Transformer bloğunun içinde, Multi-Head Attention (Çoklu Başlı Dikkat) katmanından (ve Add & Norm işleminden) hemen sonra gelen, tam bağlantılı (fully connected) standart bir yapay sinir ağıdır.
- **Transformer'lardaki FFN katmanlarının çok karakteristik bir yapısı vardır:** İlk doğrusal katman (W_1), kelimenin vektör boyutunu genellikle **4 katına çıkarır (Örn: 512 boyutlu vektörü 2048'e genişletir).** İkinci doğrusal katman (W_2) ise bunu tekrar orijinal boyutuna (512'ye) daraltır.
- **Faydası: Bu genişletme işlemi (high-dimensional projection),** modelin o kelimeyle ilgili çok daha fazla özelliği ve detayı aynı anda analiz edebilmesi için geniş bir çalışma alanı yaratır.
- FFN’ ler özelliklerin genişletilmesi ve daralması yoluyla, zengin bir keşif alanı-bilgisi oluşturmayı sağlar, FFN ler transformer hafızası olarak bilinir.

LLM architecture-Feed Forward Neural Network

FFN (Feed Forward Neural Network):

- İleri beslemeli sinir ağları:

Listing 4.4 A feed forward neural network module

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
            GELU(),
            nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
        )

    def forward(self, x):
        return self.layers(x)
```

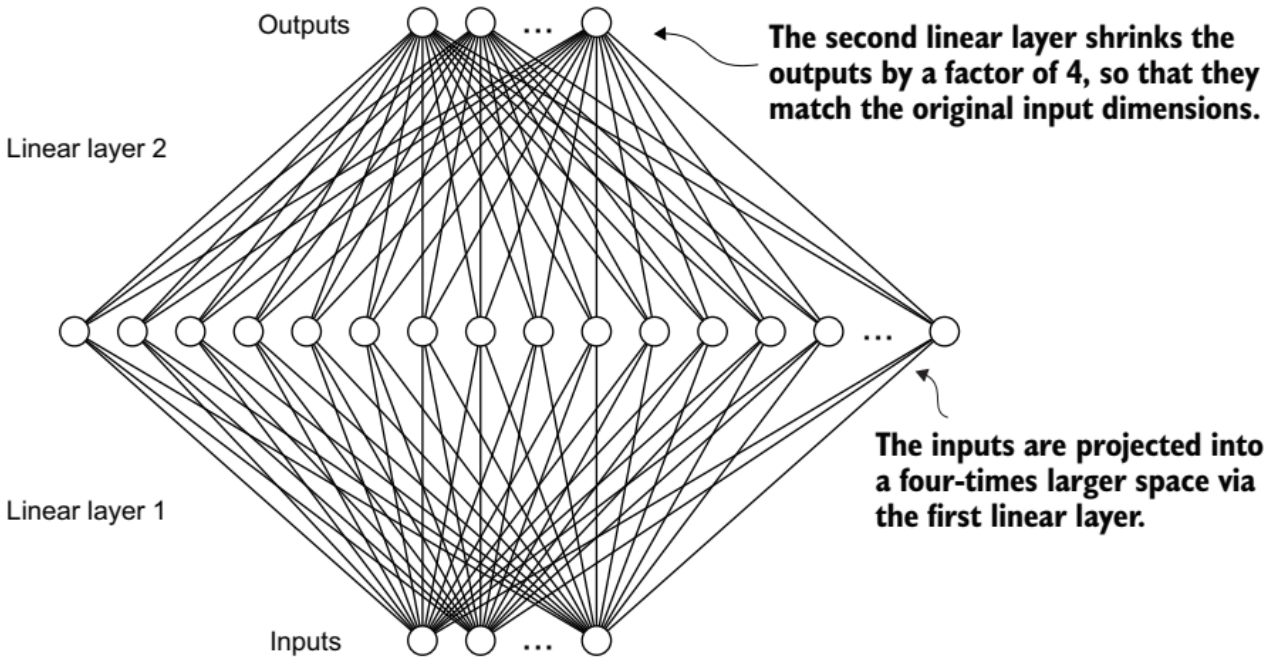


Figure 4.10 An illustration of the expansion and contraction of the layer outputs in the feed forward neural network. First, the inputs expand by a factor of 4 from 768 to 3,072 values. Then, the second layer compresses the 3,072 values back into a 768-dimensional representation.

LLM architecture-Shortcut connections

Shortcut connections:

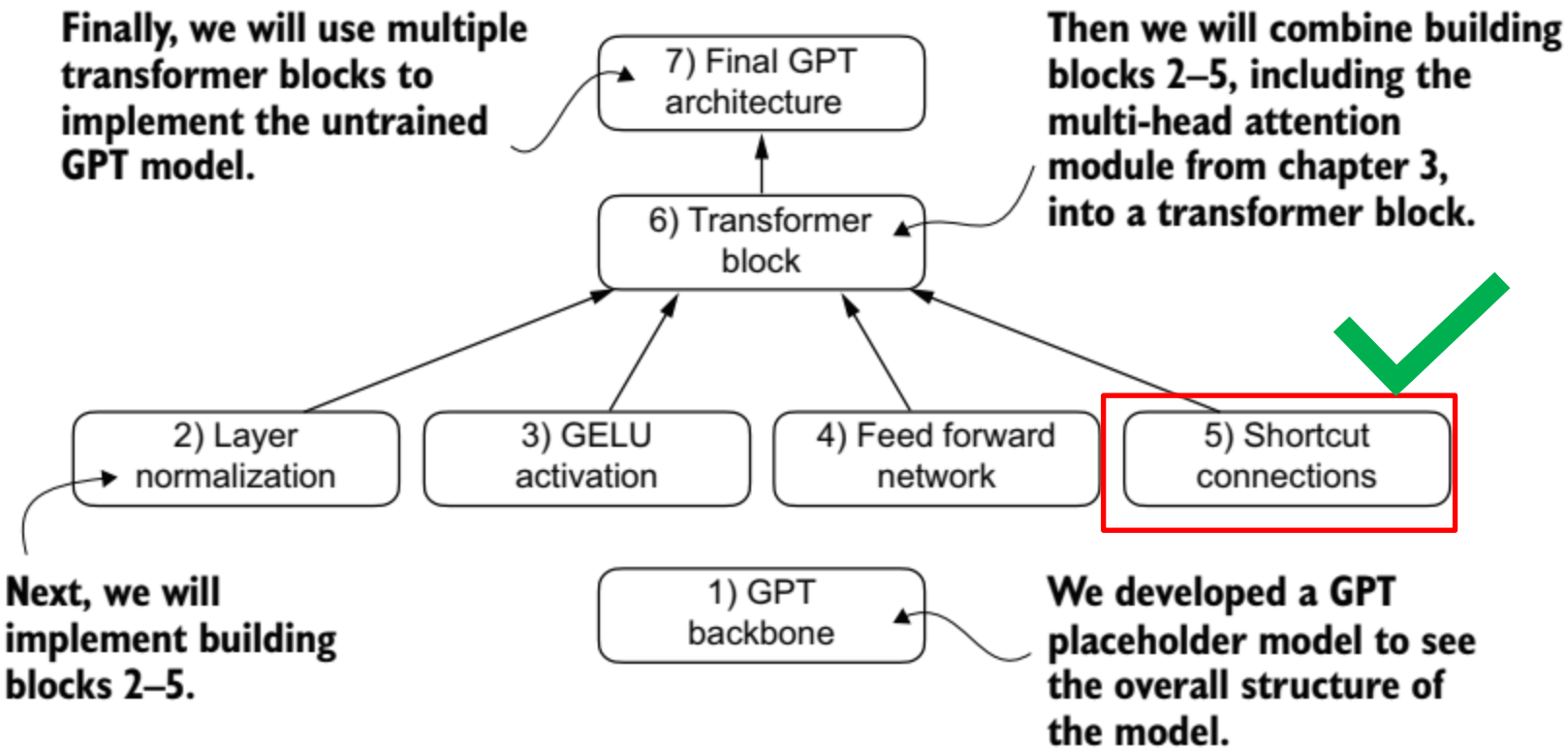
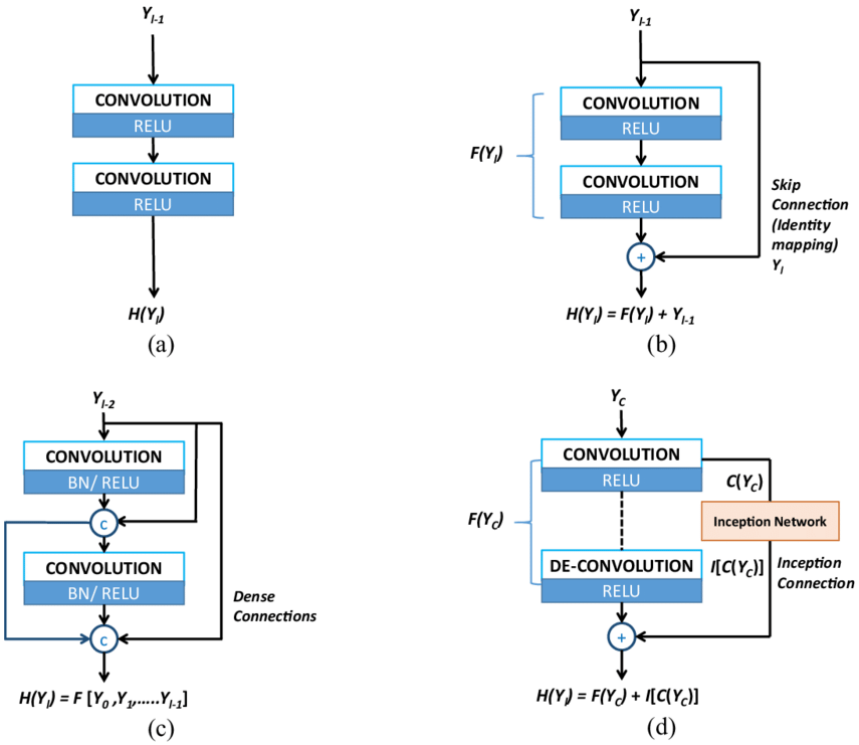


Figure 4.3 The order in which we code the GPT architecture. We start with the GPT backbone, a placeholder architecture, before getting to the individual core pieces and eventually assembling them in a transformer block for the final GPT architecture.

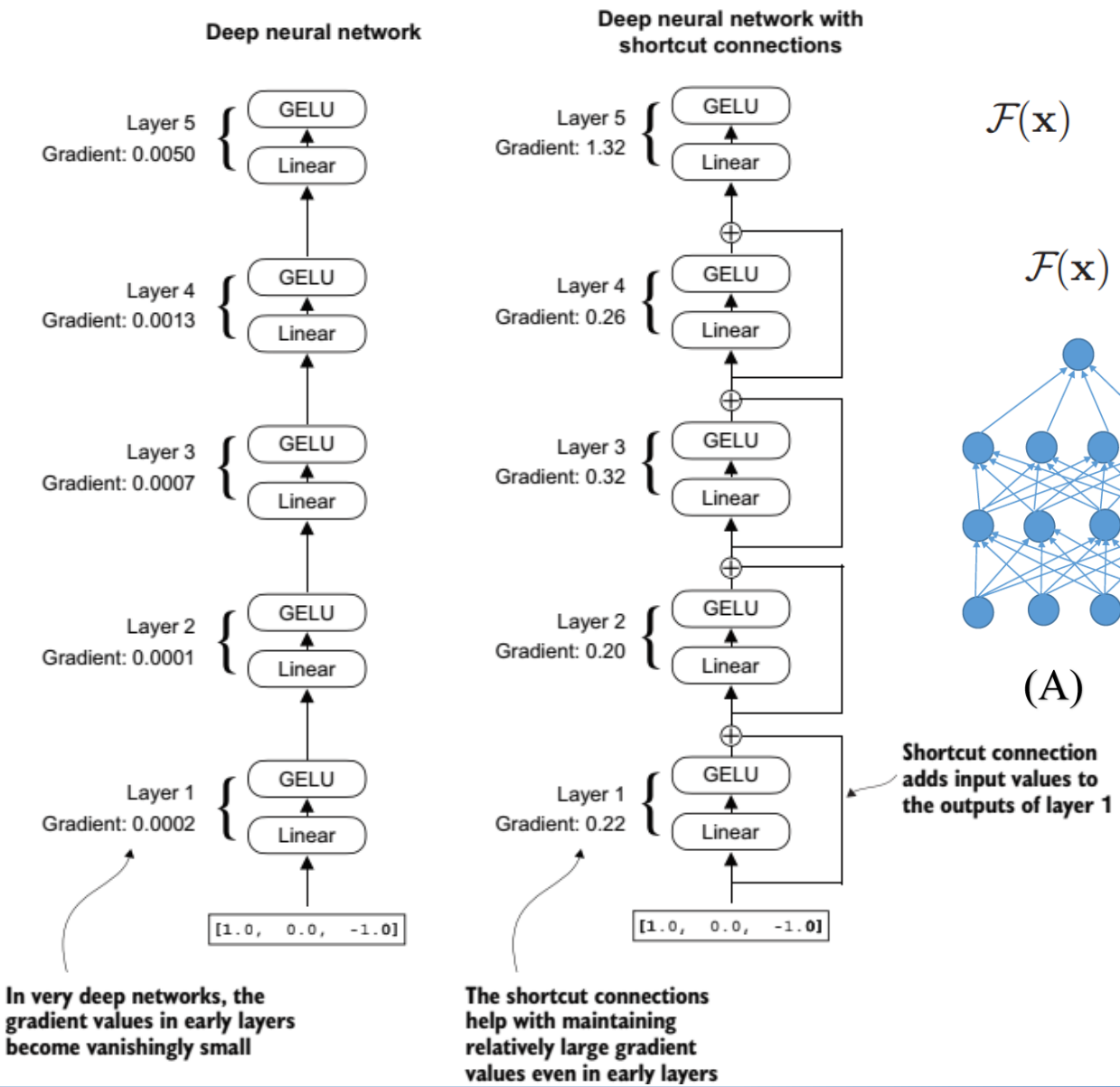
LLM architecture-Shortcut connections

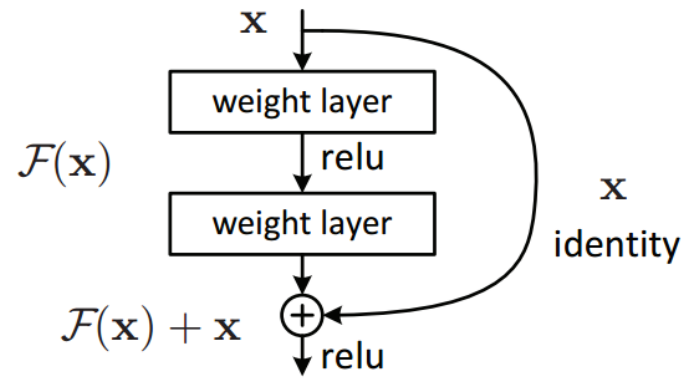
- Shortcut connections: "Shortcut Connections" (kısayol bağlantıları), makine öğrenimi literatüründe daha çok Residual Connections (Artık/Kalıntı Bağlantılar) veya Skip Connections (Atlama Bağlantıları) olarak bilinir.
- Normalde veri bir sinir ağı katmanına girer, içerideki matematiksel işlemlerden geçer ve değişmiş olarak çıkar. Kısayol bağlantısında ise verinin katmana girmeden önceki orijinal ham hali, o katmanın çıktısına doğrudan eklenir.

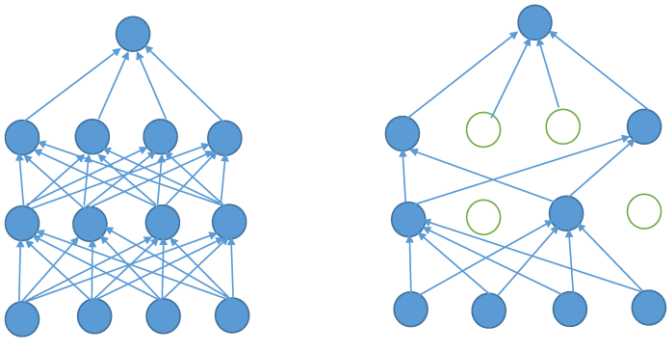


LLM architecture-Shortcut connections

- Shortcut connections:







(A) Dropout (B)

LLM architecture-Shortcut connections

Shortcut connections:

Kullanım Amacı Nedir ve Ne İşe Yarar?

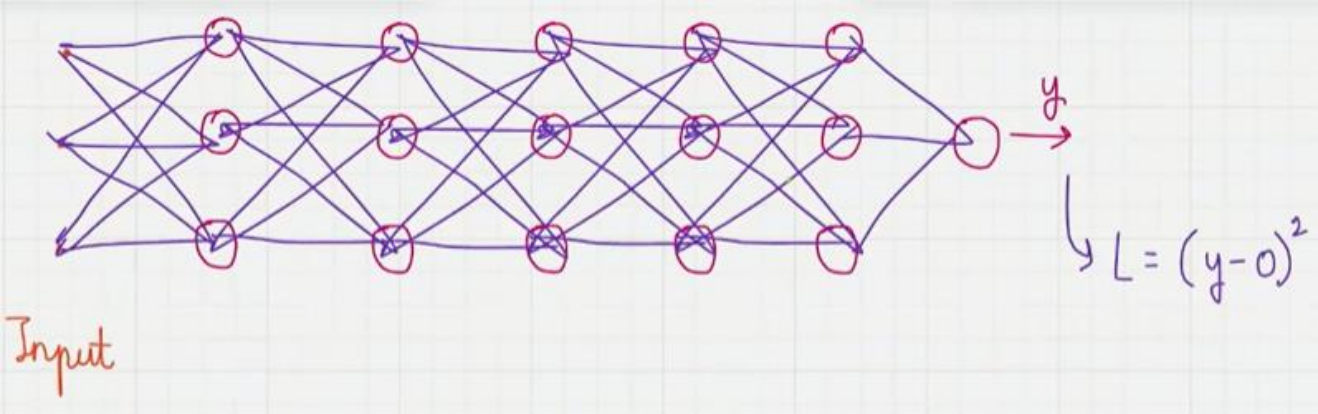
- **Kaybolan Gradyan (Vanishing Gradient) Problemini Çözer:** Yapay sinir ağları öğrenirken "geriye yayılım" (backpropagation) adı verilen bir yöntem kullanır. Hata payları ağın sonundan başına doğru iletilerek ağırlıklar güncellenir. Ancak ağ derinleştikçe (katman sayısı arttıkça), bu hata sinyalleri her katmanda bir miktar küçülür ve ilk katmanlara ulaştığında sıfıra yaklaşarak kaybolur. Bu durumda modelin ilk katmanları öğrenmeyi durdurur. Kısayol bağlantıları, bu gradyanların (hata sinyallerinin) katmanlardaki işlemleri atlayarak geriye doğru çok daha güçlü ve kesintisiz akması için bir yol oluşturur.
- **Kesintisiz Bilgi Akışı ve Orijinal Bağlamı Koruma:** Attention veya FFN katmanları veriyi işlerken, kendi görevlerine odaklandıkları için kelimelerin orijinal anlamlarına dair bazı detayları istemeden kaybedebilirler. Girdi verisini doğrudan çıktıya eklemek **(+x),** orijinal cümlenin ana bilgisinin ağın en derin ve son katmanlarına kadar güvenle taşınmasını garanti altına alır.

LLM architecture-Shortcut connections

Shortcut connections: (sonuç için kodu incele çalıştır)

```python
class ExampleDeepNeuralNetwork(nn.Module):
    def __init__(self, layer_sizes, use_shortcut):
        super().__init__()
        self.use_shortcut = use_shortcut
        self.layers = nn.ModuleList([
            nn.Sequential(nn.Linear(layer_sizes[0], layer_sizes[1]), GELU()),
            nn.Sequential(nn.Linear(layer_sizes[1], layer_sizes[2]), GELU()),
            nn.Sequential(nn.Linear(layer_sizes[2], layer_sizes[3]), GELU()),
            nn.Sequential(nn.Linear(layer_sizes[3], layer_sizes[4]), GELU()),
            nn.Sequential(nn.Linear(layer_sizes[4], layer_sizes[5]), GELU())
        ])

    def forward(self, x):
        for layer in self.layers:
            # Compute the output of the current layer
            layer_output = layer(x)
            # Check if shortcut can be applied
            if self.use_shortcut and x.shape == layer_output.shape:
                x = x + layer_output
            else:
                x = layer_output
        return x
```



LLM architecture-Shortcut connections

Shortcut connections olmadan modeli çalıştırdığımızda gradyanların değişimi.

```
print_gradients(model_without_shortcut, sample_input)
```

```
layers.0.0.weight has gradient mean of 0.00020173587836325169
layers.1.0.weight has gradient mean of 0.00012011159560643137
layers.2.0.weight has gradient mean of 0.0007152039906941354
layers.3.0.weight has gradient mean of 0.0013988736318424344
layers.4.0.weight has gradient mean of 0.005049645435065031
```

Shortcut connections uygulandığında modeli çalıştırdığımızda gradyanların değişimi.

```
torch.manual_seed(123)
model_with_shortcut = ExampleDeepNeuralNetwork(
layer_sizes, use_shortcut=True
)
print_gradients(model_with_shortcut, sample_input)
```

```
layers.0.0.weight has gradient mean of 0.22169792652130127
layers.1.0.weight has gradient mean of 0.20694106817245483
layers.2.0.weight has gradient mean of 0.32896995544433594
layers.3.0.weight has gradient mean of 0.2665732204914093
layers.4.0.weight has gradient mean of 1.3258540630340576
```

Örnek uygulama çalışması-BERT model Sentiment Analizi

Rotten Tomatoes Veri Seti

Rotten Tomatoes (movie review) veri seti, film eleştirilerinden oluşan bir **ikili duygu analizi (binary sentiment classification)** veri setidir:

- **Pozitif (1):** Filmi beğenen eleştiriler
- **Negatif (0):** Filmi beğenmeyen eleştiriler
- Her örnek **kısa bir cümle** (genelde 1-2 cümle uzunluğunda)
- Yaklaşık **5.331 pozitif** ve **5.331 negatif** olmak üzere toplam ~10.662 örnek
- HuggingFace'de hazır **train / validation / test** split'leri mevcut

Yapacağımız Mimari

```
[Metin] → BERT Tokenizer → BERT (frozen, özellik çıkarıcı) →
[CLS] embedding (768-dim)
    → MLP (3 katman, 2 output nöronu) → Softmax →
Pozitif/Negatif
```

BERT'i **eğitmiyoruz**, sadece embedding çıkarıyoruz. Sonra kendi yazdığımız PyTorch MLP'yi eğitiyoruz.

Örnek uygulama çalışması-BERT model Sentiment Analizi

1. "Rotten Tomatoes" Veri Seti Nedir? Görevi Nedir?

Rotten Tomatoes, dünyanın en popüler sinema eleştiri platformlarından biridir. Bu veri seti (dataset), film eleştirmenlerinin yazdığı kısa inceleme metinlerinden (review) oluşur.

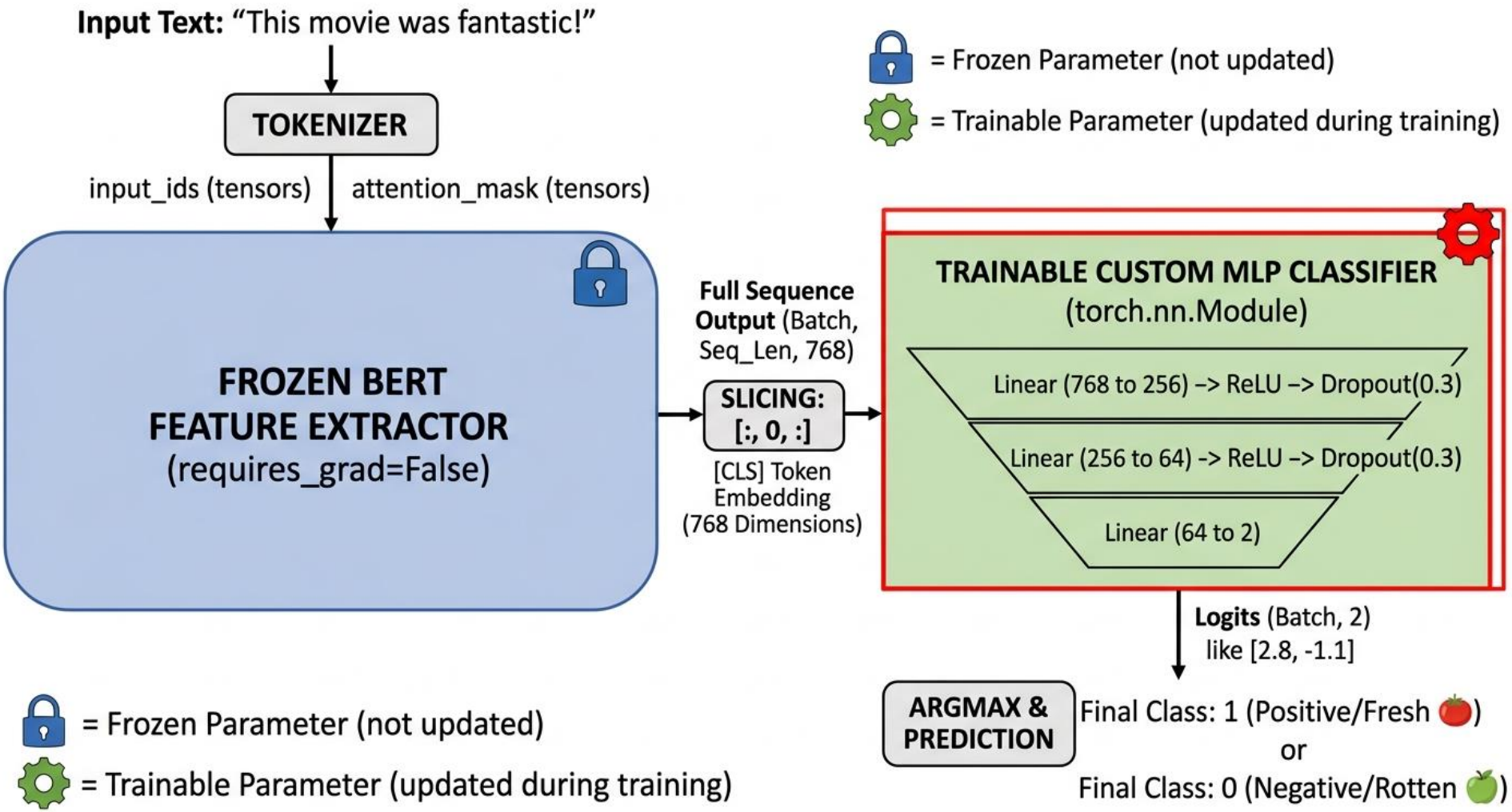
•**Veri Setinin İçeriği:** Her bir satırda bir film eleştirisi metni ve bu eleştirinin duygu durumunu belirten bir etiket (label) bulunur. Toplamda yaklaşık 10.000 civarında veri içerir.

•**Veri Tipi ve Etiketler:**

- `text`: Eleştirinin kendisi (Örn: *"Bu film hayatımda izlediğim en sıkıcı şeydi."*)
- `label`: 0 veya 1 olmak üzere iki sınıf (Binary).
  - **0 (Rotten / Çürük):** Negatif, olumsuz eleştiri.
  - **1 (Fresh / Taze):** Pozitif, olumlu eleştiri.

•**Veri Setinin Görevi:** Bu veri setinin temel makine öğrenmesi görevi **Duygu Analizi (Sentiment Analysis)**, yani bir **İkili Metin Sınıflandırma (Binary Text Classification)** işlemidir.

Örnek uygulama çalışması-BERT model Sentiment Analizi



- Doç. Dr. Musa AYDIN
- musa.aydin@samsun.edu.tr

Ofis No: 419